

# React + CSS Frontend Interview Field Guide

Rendering, async UI, streaming chat, component design, accessibility, layout, and production-quality execution

WHAT THIS IS FOR

Frontend-facing coding and architecture rounds: build a correct widget quickly, explain React's model, make it accessible, and show how the implementation survives real users and async failure.

INTERVIEW RULE

A cheatsheet should reduce recall time. It should not replace explaining the invariant, tradeoff, and failure mode in your own words.

HOW TO USE IT

- Read pages 2–3 before a practice block.
- Middle pages: only after identifying the category.
- Final checklist: during timed mocks.
- Re-code templates from memory; passive rereading is not preparation.

| \$ | FOCUS                        | USE IT FOR                                                    |
|----|------------------------------|---------------------------------------------------------------|
| 02 | Render + state               | snapshots, updates, identity, keys, forms                     |
| 03 | Effects + refs               | external sync, cleanup, custom hooks                          |
| 04 | Async UI                     | fetch, cancellation, races, Suspense, optimistic UI           |
| 05 | Streaming chat I <b>NEW</b>  | transport choice, reader loop, SSE frame parsing              |
| 06 | Streaming chat II <b>NEW</b> | status machine, stop/retry, chat UX — the OpenAI-screen build |
| 07 | Component design             | state ownership, context, reducer, performance                |
| 08 | Widget execution             | a11y, tabIndex, keyboard, testing, definition of done         |
| 09 | CSS layout                   | box model, flex, grid, position, scroll                       |
| 10 | CSS systems                  | cascade, responsive, tokens, motion                           |
| 11 | TypeScript + SSR             | types, hydration, React 19 awareness                          |
| 12 | Final drill                  | debugging tree, checklist, references                         |

## 02 React mental model: render, state, identity

### A. RENDER → COMMIT → EFFECT

- **Render** calculates JSX from a **snapshot** of props/state. It must be pure: no mutation, subscriptions, timers, or imperative DOM work.
- **Commit** applies DOM changes and refs. Effects run afterward; layout effects run synchronously after DOM mutation, before paint.
- A state setter **schedules a render**. The current handler still sees its render's snapshot; use functional updates when the next value depends on the previous.

```
setCount(c => c + 1);           // safe under queued
updates
setItems(xs => [...xs, item]);
setUser(u => ({...u, name}));
```

### B. STATE DESIGN

- Keep **minimal state**. Derive filtered lists, totals, labels, and validity during render unless computation is genuinely expensive.
- Choose **one owner** for each piece of state. Lift to the nearest common owner; colocate state that no sibling needs.
- Do not mirror props into state unless you are intentionally taking an editable snapshot and can define reset semantics.
- Use a reducer when transitions/invariants are clearer as **events**, not merely because there are several fields.

### C. IDENTITY AND KEYS

- React preserves state by **component type + position** in the rendered tree. A changed **key** intentionally resets that subtree.
- A list key must be **stable among siblings** and represent the entity. Index is unsafe when insert/delete/reorder can occur.
- Keys are not props and need only be locally unique. Generate IDs when data is **created**, not during render.

- Defining a component function inside another component creates a new type each render and can reset state.

### D. CONTROLLED FORMS

```
const [form, setForm] = useState({email:'', pw:''});
const change = e => setForm(f => ({
  ...f, [e.target.name]: e.target.value
}));

<form onSubmit={submit}>
  <label>Email
    <input name="email" value={form.email}
      onChange={change}/>
  </label>
  <button>Submit</button> // type=submit by default
</form>
```

- **Controlled:** React state is source of truth. **Uncontrolled:** DOM holds the current value via **defaultValue**/ref. Do not switch mid-lifecycle.
- In the submit handler, call **e.preventDefault()** before async work; a full-page form post is the classic silent bug.
- Model validation states deliberately: pristine, invalid, submitting, server error, success. Validate on blur or submit first, then on change once a field has erred. Prevent double submit.

### E. COMPONENT BOUNDARIES

- Extract when a part has its own responsibility/state, is reusable/testable, or isolates expensive rendering.
- Prefer data + callbacks/children over giant boolean prop matrices. Use composition before context.
- A custom hook shares **stateful logic, not state itself**; each call has independent state unless it subscribes to an external store.

#### SAY THIS

"I am keeping state minimal and colocated. Derived values stay in render; side effects only synchronize with systems outside React."

## 03 Effects, refs, and external systems

### A. YOU MIGHT NOT NEED AN EFFECT

| NEED                          | BETTER PLACE               |
|-------------------------------|----------------------------|
| Derive value from props/state | render                     |
| Respond to click/submit       | event handler              |
| Reset subtree for entity      | key                        |
| Notify parent of input        | same event that changes it |
| Cache expensive pure value    | useMemo after profiling    |
| Sync network/widget/timer     | effect                     |

### B. EFFECT CONTRACT

- An effect **starts synchronization** with an external system; cleanup stops it. It may start/stop many times, not only mount/unmount.
- Every reactive value read belongs in dependencies. Fix unstable objects/functions or restructure; do not silence the linter.
- Strict Mode development intentionally runs an extra setup/cleanup cycle to expose missing cleanup. Write idempotent synchronization.

```
useEffect(() => {
  const ctrl = new AbortController();
  load(id, {signal: ctrl.signal}).then(setData, e => {
    if (e.name !== 'AbortError') setError(e);
  });
  return () => ctrl.abort();
}, [id]);
```

### C. REF VS STATE

| REF                                 | STATE                       |
|-------------------------------------|-----------------------------|
| mutable <code>.current</code> value | rendered snapshot           |
| change does not render              | setter schedules render     |
| DOM node, timer, latest handle      | anything visible to UI      |
| escape hatch                        | declarative source of truth |

- Do not read/write refs during render except predictable initialization. Use callback refs for dynamic node collections.

- `useLayoutEffect` only for measurement/imperative layout that must happen before paint; it blocks paint.

### D. EVENTS VS EFFECTS

- An **event handler** is caused by a specific interaction and does not rerun because dependencies changed.
- An **effect** is caused by rendering with a set of reactive values and re-synchronizes when those values change.
- Keep purchase/send/log **actions** in the handler. Keep connection/subscription **synchronization** in the effect.

### E. CUSTOM HOOK DESIGN

- Name `useX`; accept declarative inputs; return data/status/actions; hide cleanup and race handling.
- Keep the API smaller than the implementation. Avoid returning unstable object/function identities unless harmless.
- A hook that subscribes to an external mutable store should use the external-store contract (`useSyncExternalStore`, §07.C) rather than ad-hoc effect + `setState`.

### F. PORTALS AND IMPERATIVE HANDLES

- A portal changes **DOM placement, not React ownership**/event propagation. Useful for modal/popover escape from clipping/stacking.
- Refs expose imperative focus/scroll/measure. Keep the surface narrow and prefer declarative props.

#### EFFECT DEBUG

If an effect loops, ask: what external system am I synchronizing with? Which dependency changes *because this effect sets state*? Can the value be derived, or the action moved to an event?

## 04 Async UI, races, and concurrency

### A. ASYNC STATE MACHINE

- Model `idle` → `pending` → `success` | `error`; optionally refreshing, streaming, canceled, partial, empty.
- **Ignore/abort stale results** when query identity changes. A late response must not overwrite newer data.
- Separate initial loading from background refresh so useful content does not disappear.
- Error state should preserve retry context and distinguish validation, offline, auth, rate-limit, server, and parse failures when useful.

### B. FETCH HOOK SKELETON

```
function useResource(url) {
  const [s, setS] = useState({status:'pending'});
  useEffect(() => {
    const c = new AbortController();
    setS({status:'pending'});
    fetch(url, {signal: c.signal}).then(r => {
      if (!r.ok) throw Error(String(r.status));
      return r.json();
    }).then(
      data => setS({status:'success', data}),
      e => { if (e.name !== 'AbortError')
        setS({status:'error', error:e}); }
    );
    return () => c.abort();
  }, [url]);
  return s;
}
```

For real applications, a data library may own caching, dedupe, retries, invalidation, and SSR. In an interview, show you know the missing concerns.

### C. SUSPENSE AND ERROR BOUNDARIES

- A Suspense boundary owns the pending fallback for code/data sources integrated with Suspense. An error boundary owns render/async-action errors it can catch.
- Place boundaries around **meaningful UX regions**. Avoid replacing the whole page for one slow panel.
- Lazy-loaded components suspend while code loads. Hydration output must match server HTML.

### D. TRANSITIONS AND DEFERRED VALUES

- Urgent updates keep inputs responsive. A transition marks non-urgent rendering that may be interrupted/restarted.
- `useTransition` when **you trigger** the update and need pending state; `useDeferredValue` when **consuming** a value more slowly.
- Transitions do not make network calls faster and are not for controlled input state itself.

### E. OPTIMISTIC UI

- Apply predicted state with a stable operation ID; reconcile the server response; roll back or show conflict on failure.
- Safe when reversal is understandable. Be cautious for money, inventory, permissions, and irreversible effects.

#### SAY THIS

"I will keep the input urgent, cancel stale work, preserve current content during refresh, and put pending/error boundaries at the smallest useful UX region."

## 05 Streaming chat I: transport and SSE parsing

### A. TRANSPORT DECISION

| TRANSPORT                          | USE WHEN                                                                                                                                  |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>fetch POST + ReadableStream</b> | authenticated request/response streams; you send a body (chat). You own parsing, retry, reconnect.                                        |
| <b>EventSource (SSE)</b>           | browser-managed <b>GET</b> stream: auto-reconnect with <a href="#">Last-Event-ID</a> , named events, but no POST body, no custom headers. |
| <b>WebSocket</b>                   | bidirectional (typing indicators, multiplayer). Overkill for one-way token streams.                                                       |

Chat APIs need a POST body, so the interview answer is almost always **fetch + reader loop** — and saying *why* EventSource doesn't fit is the signal.

### B. READER LOOP + SSE FRAME PARSING

```
async function streamChat(messages, signal, onToken) {
  const res = await fetch('/api/chat', {
    method: 'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({messages}),
    signal,
  });
  if (!res.ok || !res.body) throw
  Error(String(res.status));

  const reader = res.body.getReader();
  const dec = new TextDecoder();
  let buf = ''; // partial frames
  survive chunks
  while (true) {
    const {done, value} = await reader.read();
    if (done) break;
    buf += dec.decode(value, {stream: true});
    // stream:true keeps split multi-byte chars intact
    const frames = buf.split('\n\n'); // SSE frames end
    buf = frames.pop(); // last piece may be
    incomplete
    for (const f of frames) {
      for (const line of f.split('\n')) {
        if (!line.startsWith('data:')) continue;
        const data = line.slice(5).trim();
        if (data === '[DONE]') return;
        onToken(JSON.parse(data)); // try/catch in real
      }
    }
  }
}
```

**Invariants to narrate:** chunk boundaries do not align with frame boundaries (buffer the tail); `{stream:true}` so split multi-byte UTF-8 decodes correctly; `[DONE]` or stream close is the terminal signal.

## 06 Streaming chat II: component state and chat UX

### A. COMPONENT STATE MACHINE

```
const [status, setStatus] = useState('idle');
// idle|streaming|done|error|stopped
const ctrlRef = useRef(null);

async function send(text) {
  ctrlRef.current?.abort(); // kill any prior
  stream
  const ctrl = new AbortController();
  ctrlRef.current = ctrl;
  const userMsg = {role:'user', content:text};
  setMsgs(m => [...m, userMsg,
    {role:'assistant', content:''}]);
  setStatus('streaming');
  try { // msgs = this render's snapshot; abort covers
    await streamChat([...msgs, userMsg], ctrl.signal, tok
=>
    setMsgs(m => { // append to LAST msg
      only
      const out = m.slice();
      const last = out[out.length - 1];
      out[out.length-1] = {...last,
        content: last.content + tok};
      return out;
    }));
    setStatus('done');
  } catch (e) {
    if (e.name === 'AbortError') setStatus('stopped');
    else
    setStatus('error'); // keep partial text + retry
    ctx
  }
}
const stop = () => ctrlRef.current?.abort();
useEffect(() => () => ctrlRef.current?.abort(), []);
```

- **Stop ≠ error:** user abort keeps the partial message and re-enables input; a network error offers retry with the same request.

- One send button doubles as **Stop** while streaming — disabling input entirely reads junior.
- Retry must not re-append the user message — resend from existing state.

### B. CHAT UX DETAILS THAT SCORE

- **Render batching:** per-token `setState` is usually fine (React batches); if asked about jank, accumulate in a ref and flush on rAF.
- **Pinned-to-bottom scroll:** auto-scroll on new tokens *only* if the user is already near the bottom (track via scroll position or a sentinel + IntersectionObserver). Scrolling a reading user is a classic fail.
- `aria-live` per token is noise — announce on completion, or `role="log"` on the transcript. Keys: message IDs generated at creation, never index (retry/regenerate reorders).
- Deferred seams to name: markdown rendering, virtualized history, persistence, reconnect-with-cursor.

#### SAY THIS

"I'll buffer partial SSE frames across chunks, drive the UI from a status machine, make Stop a first-class state keeping partial output, and only auto-scroll when the user is at the bottom."

## 07 Component and state architecture

### A. STATE OWNERSHIP DECISION

| STATE                            | OWNER                         |
|----------------------------------|-------------------------------|
| Input draft                      | field/form component          |
| Selected item shared by siblings | nearest common parent         |
| Server resource/cache            | data layer                    |
| URL-shareable filter             | router/URL                    |
| Theme/auth/locale                | stable context/external store |
| Ephemeral hover/open             | local component               |

### B. REDUCER FOR EXPLICIT TRANSITIONS

```
function reducer(s, a) {
  switch (a.type) {
    case 'submit': return {...s, status:'pending'};
    case 'success': return {...s, status:'done',
data:a.data};
    case 'failure': return {...s, status:'error',
error:a.error};
    default: throw Error('unknown action');
  }
}
```

- A reducer must be pure. Actions describe **events**, not **setters**. Discriminated-union action types make invalid payloads unrepresentable.

### C. CONTEXT WITHOUT RENDER STORMS

- Every consumer rerenders when provider **value identity** changes. Memoizing the value helps only if its dependencies are stable.
- Split contexts by update frequency/responsibility; pass children/props when scope is narrow.
- For high-frequency external state, subscribe with selectors:

```
const value = useSyncExternalStore(
  store.subscribe, // (cb) => unsubscribe
  () => store.get(), // client snapshot (must be cached)
  () => serverSnapshot // SSR snapshot (optional)
);
```

### D. PERFORMANCE: PROFILE FIRST

- Fix architecture before memoization: move state down, split components, avoid effect chains, virtualize large lists, cache server data.
- `memo` skips when props compare equal; `useMemo` caches a value; `useCallback` caches function identity. All add complexity.
- A single always-new object/function can defeat memoization. React Compiler may reduce manual memo needs in configured projects; still understand identity.
- Measure commit duration, interaction latency, rerender cause, list size, and expensive computation.

### E. COMPONENT API QUALITY

- Prefer semantic variants ( `tone="danger"` ) over presentation knobs when building reusable UI.
- Controlled/uncontrolled open state: document precedence and callback timing. Avoid half-controlled ambiguity.
- Forward accessible name, disabled/loading state, refs, and DOM attributes deliberately. Do not leak internal state shape.
- Use render props/compound components only when they improve composition enough to justify the API complexity.

### F. ERROR AND RECOVERY DESIGN

- Render errors → error boundary. Event-handler/async errors → catch and set state / report explicitly.
- Retry must not duplicate side effects. Preserve input where possible. Log component/operation context.

#### STAFF SIGNAL

Explain who owns each state, how updates propagate, which subtree rerenders, how async races are prevented, and where the API can evolve without rewriting consumers.

## 08 Timed widget execution: accessibility and testing

### A. THE 45-MINUTE FRONTEND LOOP

| MIN   | DO                                                                                            |
|-------|-----------------------------------------------------------------------------------------------|
| 0-4   | clarify interactions, data shape, keyboard behavior, responsive target, required async states |
| 4-8   | semantic HTML and state model; name source of truth and component boundaries                  |
| 8-25  | happy path end to end; simple CSS layout, real controls                                       |
| 25-35 | loading/empty/error, keyboard/focus, cleanup/races, rapid interaction                         |
| 35-42 | tests and edge cases                                                                          |
| 42-45 | explain tradeoffs and production extensions                                                   |

### B. ACCESSIBILITY DEFAULTS

- Use native `button`, `input`, `select`, `label`, `dialog`, `table`, `ul/ol` before recreating semantics with `div` + ARIA. **No ARIA is better than bad ARIA.**
- Every control has an accessible name. Label inputs by nesting or matched IDs — `useId` makes this SSR-safe:

```
const id = useId();
<label htmlFor={id}>Email</label>
<input id={id} aria-describedby={id+'-err'}
  aria-invalid={!error}/>
{error && <p id={id+'-err'} role="alert">{error}</p>}
```

- Keyboard: Tab order follows DOM; Enter/Space activate buttons; Escape closes transient layers; arrow keys only for composite widget patterns (APG).
- Visible focus with `:focus-visible`. Disabled means truly disabled, or intentionally `aria-disabled` with behavior blocked.
- Announce async status sparingly with `role="status" / aria-live="polite"`. Do not move focus for every update.

### C. TABINDEX: THE THREE-VALUE MODEL

| VALUE  | MEANING                                                                                                |
|--------|--------------------------------------------------------------------------------------------------------|
| (none) | native focusability; interactive elements are already in tab order                                     |
| 0      | adds element to natural tab order (custom widget hosts)                                                |
| -1     | focusable via JS <code>.focus()</code> only — roving tabindex members, dialog containers, skip targets |
| > 0    | hijacks global tab order — effectively never use                                                       |

- **Roving tabindex** (toolbar/tabs/listbox): one member has `tabIndex=0`, the rest `-1`; arrow keys move both focus and the 0.

### D. MODAL / POPOVER CHECKLIST

- Use native `<dialog>` when allowed, or implement: dialog role + name, initial focus, focus containment, Escape, restore trigger focus, inert background.
- Portal to the top layer/appropriate root; prevent background scroll; account for nested layers and outside-click races.
- Do not make blanket claims that Tab never leaves; state the chosen containment behavior.

### E. TEST BEHAVIOR, NOT IMPLEMENTATION

- Query as a user: role + accessible name. Exercise click, typing, keyboard, focus, async completion, and error.
- Test visible outcome and callbacks, not hook calls/internal state. Use fake timers carefully for debounce.
- Async test: pending visible → outcome; stale response does not win; unmount/abort does not warn or update.
- One integration test for the full widget catches more than snapshots of each child.

### F. DEFINITION OF DONE

- Happy path; empty/loading/error/disabled states; rapid repeat; cleanup; stable keys.
- Keyboard, focus, names, contrast, reduced motion, responsive layout, long content.
- No state mutation, stale-response win, duplicate submit, missing key, or effect loop.
- Explain performance only after correctness; mention virtualization/caching if scale warrants.

#### INTERVIEW MOVE

Name one deferred production concern explicitly — virtualization, i18n, analytics, optimistic reconciliation, cross-browser testing — then finish the requested behavior before expanding scope.



## 09 CSS layout and debugging

### A. BOX MODEL AND SIZING

```
*, ::before, ::after { box-sizing: border-box; }
img { max-width: 100%; height: auto; }
.shell { width: min(100% - 2rem, 72rem);
  margin-inline: auto; }
```

- Block size includes content + padding + border unless `border-box`. Margins can collapse vertically in normal block flow.
- Percent height needs a definite containing height. Prefer `min-height: 100dvh` for viewport shells, with mobile caveats.
- `min-width: 0` lets flex/grid children shrink; `minmax(0, 1fr)` prevents content overflow.

### B. FLEXBOX: ONE DIMENSION

- Main axis from `flex-direction`; `justify-content` aligns the main axis, `align-items` the cross axis; `gap` for spacing.
- `flex: grow shrink basis`. `flex: 1` commonly behaves as equal flexible items, but intrinsic minimum size can block shrink.
- Use `margin-inline-start: auto` to push one item. Use `flex-wrap` for rows that may wrap.

```
.row { display: flex; align-items: center; gap: .75rem; }
.row__main { flex: 1; min-width: 0; }
.row__actions { flex: none; }
```

### C. GRID: TWO DIMENSIONS

```
.cards { display: grid; gap: 1rem;
  grid-template-columns:
    repeat(auto-fit, minmax(min(16rem, 100%), 1fr)); }
.layout { display: grid;
  grid-template-columns: 16rem minmax(0, 1fr); }
```

- Grid places in rows and columns; flex lays out one axis. `subgrid` aligns nested tracks when supported by target browsers.

- `auto-fit` collapses empty tracks; `auto-fill` preserves them. Fraction units distribute **remaining** space, not intrinsic minimums.

### D. POSITIONING AND STACKING

- `absolute` positions against the nearest **positioned** containing block; `fixed` usually the viewport; `sticky` needs a scroll container and an inset.
- `z-index` compares **within stacking contexts**. Transform, opacity, isolation, positioned z-index, and others create new contexts.
- If a modal cannot rise above a sibling, inspect stacking-context ancestry or portal/top layer — do not keep increasing z-index.

### E. OVERFLOW AND SCROLL

- **Choose the scroll owner.** Nested overflow can break sticky, wheel behavior, focus visibility, and mobile height.
- Text truncation: `min-width: 0; overflow: hidden; text-overflow: ellipsis; white-space: nowrap`. Multi-line needs a line-clamp/support decision.
- Reserve scrollbar/layout space if shifts matter. Keep focused content visible; use `scroll-padding` for sticky headers.

### F. DEBUG ORDER

- Inspect computed value and winning rule → containing block → intrinsic min size → overflow owner → stacking context → browser support.
- Use temporary outlines and disable rules. Do not solve a layout constraint with arbitrary pixels until you can name it.

## 10 CSS cascade, responsive systems, and motion

### A. CASCADE ORDER

- Origin/importance → cascade layer → specificity → scope proximity → source order. Specificity only compares rules still competing.
- Specificity weights conceptually: IDs > classes/attributes/pseudo-classes > elements/pseudo-elements. `:where()` adds zero specificity.
- Use cascade layers and low-specificity component selectors; avoid `!important` escalation.

```
@layer reset, base, components, utilities;
@layer components { .button { ... } }
@layer utilities { .srOnly { ... } }
```

### B. RESPONSIVE BY CONSTRAINT

- Start fluid: `max/min/clamp`, wrapping, flexible tracks. Add a breakpoint where the **content fails**, not for a named device.
- A media query responds to viewport/user preference. A container query responds to the component's available space.

```
.cardShell { container-type: inline-size; }
@container (width >= 32rem) {
  .card { grid-template-columns: 8rem 1fr; }
}
h1 { font-size: clamp(1.5rem, 1rem + 2vw, 3rem); }
```

### C. TOKENS AND THEMES

- CSS custom properties inherit and can change at runtime. Define **semantic tokens**, not only raw palette values.

```
:root { --surface:#fff; --text:#172033; --space-3:.75rem; }
[data-theme='dark'] { --surface:#101828; --text:#f8fafc; }
.card { background:var(--surface); color:var(--text); }
```

- Check contrast, forced-colors/high-contrast, and system color scheme according to product support.

### D. MOTION

- Animate opacity/transform when possible; layout properties may trigger repeated layout/paint.
- Motion communicates relationship/state, not decoration alone. Keep duration and interruption behavior consistent.

```
@media (prefers-reduced-motion: reduce) {
  *, ::before, ::after {
    scroll-behavior:auto; animation-duration:.01ms;
    animation-iteration-count:1; transition-duration:.01ms; }
}
```

### E. SEMANTIC CSS ARCHITECTURE

- Choose one approach consistent with the codebase: modules, CSS-in-JS, utilities, BEM, or layered global CSS.
- Keep component styles local, tokens global, states explicit, and specificity flat. Avoid selectors coupled to incidental DOM depth.
- Inline styles are useful for dynamic values but do not replace pseudo-classes, media/container queries, or theme structure.

#### CSS INTERVIEW MOVE

Describe the layout constraint first: one axis or two, intrinsic sizing, scroll owner, and the responsive failure point. Then choose flex/grid/positioning.

## 11 TypeScript, SSR/hydration, and modern React awareness

### A. TYPESCRIPT FOR COMPONENTS

```
type Props = {
  item: Item; selected?: boolean;
  onSelect(id: Item['id']): void;
  children?: React.ReactNode;
};
type State =
  | {status:'idle'}
  | {status:'pending'}
  | {status:'success'; data: Data}
  | {status:'error'; error: Error};
```

- Discriminated unions encode async states better than independent booleans. Narrow with `switch` and an exhaustiveness check.
- Use `unknown` at untrusted boundaries and validate/narrow. Avoid `any`. Prefer inference for local values.
- Events: `ChangeEvent`, `FormEvent`. Ref types include `null`.

### B. GENERICS AND UTILITIES

```
function List<T extends {id:string}>({items, render}: {
  items: T[]; render(item: T): React.ReactNode
}) {
  return <ul>{items.map(x =>
    <li key={x.id}>{render(x)}</li>)}</ul>;
}
const config = {...} satisfies Config;
```

- Useful utilities: `Pick`, `Omit`, `Partial`, `Required`, `Record`, `Readonly`, `ReturnType`, `Awaited`. Do not make every component generic.

### C. SSR AND HYDRATION

- Server render produces HTML; hydration attaches client behavior. Initial client output must match server output.
- Common mismatches: Date/random/browser-only APIs, invalid HTML nesting, different data, conditional `window` checks in render.

- Move browser-only synchronization to an effect, seed deterministic data, and preserve IDs with `useId` for label relationships.
- Streaming SSR/Suspense can reveal regions as ready; boundary design is user experience and failure isolation.

### D. REACT 19 AWARENESS

- Know concepts, not trivia: async actions/transitions, form status/actions, optimistic state, `use` with Suspense-capable resources, ref-handling changes.
- Do not force experimental/new APIs into a CoderPad prompt. Confirm project/runtime and write portable React when uncertain.
- React Compiler may automate memoization in configured builds; manual identity knowledge still matters for APIs/effects/external libraries.

### E. SERVER VS CLIENT BOUNDARY

- Keep secrets/data access and heavy non-interactive work on the server. Client components own browser APIs and interaction.
- Props across a server/client boundary must be serializable per the framework contract. Avoid shipping unnecessary code/data.
- Caching and revalidation semantics belong to the framework/data layer, not a universal React hook rule.

### F. TESTING TYPES

| TEST        | BEST FOR                        |
|-------------|---------------------------------|
| Unit        | pure reducer/formatter/utility  |
| Component   | widget behavior + accessibility |
| Integration | data/router/form flow           |
| E2E         | critical browser journey        |
| Visual      | layout/theme regression         |
| Performance | interaction/render budget       |

## 12 Final drill

### A. BUG → FIRST QUESTION

| SYMPTOM                    | CHECK                                                           |
|----------------------------|-----------------------------------------------------------------|
| Old value in handler       | render snapshot; functional update                              |
| Effect loops               | unnneeded effect; unstable dependency                           |
| Wrong row state            | unstable/index key                                              |
| Late request overwrites    | abort / request identity                                        |
| Garbled stream text        | decoder missing <code>stream:true</code> ; frame buffer dropped |
| Child rerenders            | state placement; prop identity; profile                         |
| Sticky fails               | scroll ancestor / inset                                         |
| Ellipsis fails             | <code>min-width:0</code> + overflow chain                       |
| z-index loses              | stacking context / top layer                                    |
| Keyboard cannot use widget | native semantic / APG pattern                                   |
| Hydration warning          | nondeterministic initial render                                 |

### B. FINAL FIVE-MINUTE CHECK

- Semantic structure and accessible names; keyboard/focus behavior.
- Minimal state, stable keys, immutable updates, no derived-state effect.

- Loading/empty/error/disabled/success; stale request canceled; cleanup.
- Responsive long-content layout; reduced motion; visible focus.
- Tests for primary interaction, failure, and rapid repeat.
- Explain a production seam: data cache, virtualization, i18n, analytics, monitoring.

### C. PRIMARY REFERENCES

- **React docs** — state snapshots, preserving/resetting state, effects, transitions, Suspense, external stores: [react.dev](https://react.dev)
- **MDN CSS** — flexbox, grid, cascade/specificity, stacking contexts, container queries, focus-visible, reduced motion: [developer.mozilla.org/docs/Web/CSS](https://developer.mozilla.org/docs/Web/CSS)
- **WAI-ARIA Authoring Practices** — widget keyboard and focus patterns: [w3.org/WAI/ARIA/apg/](https://w3.org/WAI/ARIA/apg/)
- **TypeScript Handbook** — narrowing, generics, utility types, satisfies: [typescriptlang.org/docs/handbook/](https://typescriptlang.org/docs/handbook/)

#### FINAL MOVE

Finish a small, correct, accessible end-to-end widget before adding abstractions. Then explain how you would evolve data fetching, scale, testing, and observability in production.